GRUPO DE LECTURA · ARXIV V5

Aprender a decidir

Guía de *Reinforcement Learning: An Overview*, de Kevin P. Murphy. Conocimientos previos, resúmenes y ocho laboratorios en CPU, con resultados y preguntas para discutir.

Las explicaciones y experimentos son una adaptación didáctica propia. Siete capítulos técnicos y un cierre metodológico para los agradecimientos.

Conocimientos previos
Capítulo 1 · Introducción
Capítulo 2 · Métodos basados en valores
Capítulo 3 · Métodos basados en políticas
Capítulo 4 · Métodos basados en modelos
Capítulo 5 · Aprendizaje por refuerzo multiagente
Capítulo 6 · LLM y aprendizaje por refuerzo
Capítulo 7 · Otros temas de RL
Capítulo 8 · Agradecimientos y práctica de cierre
Plan de lectura y glosario
Protocolo de reproducción e interpretación
Resultados observados
Fuente, versión y atribución
Instrucciones y cuadernos

Conocimientos previos

Esta guía asume Python básico y repasa las matemáticas necesarias. No hace falta conocer RL antes de empezar. La prioridad es comprender las expectativas, el condicionamiento y los gradientes; las demostraciones de convergencia pueden quedar para una segunda lectura.

1. Probabilidad: pensar en resultados posibles

Una variable aleatoria representa un resultado incierto. Para una recompensa discreta, su esperanza es

$$\mathbb{E}[R] = \sum_r p(r) r .$$

Una acción que entrega 10 con probabilidad 0.2 y 0 en el resto de casos tiene recompensa esperada 2. Esto no significa que entregue 2 en una ejecución. La varianza mide la dispersión alrededor de la media:

$$\mathrm{Var}\,(R) = \mathbb{E}[(R - \mathbb{E}[R])^2] .$$

La distribución condicional $p(s' \mid s, a)$ describe qué puede pasar después de actuar. La ley de la esperanza total permite separar decisiones y consecuencias:

$$\mathbb{E}[R \mid s] = \sum_a \pi(a \mid s) \sum_{s'} p(s' \mid s, a) \mathbb{E}[R \mid s, a, s'] .$$

Ejercicio: una política elige A el 75% de las veces y B el 25%. Sus recompensas esperadas son 2 y 6. La recompensa esperada de la política es $0.75(2) + 0.25(6) = 3$.

Monte Carlo significa aproximar una esperanza mediante una media de muestras. Con muestras independientes y varianza finita, el error estándar de la media disminuye como $1/\sqrt{n}$. Multiplicar por cuatro las muestras reduce ese error aproximadamente a la mitad. Las transiciones consecutivas de una misma trayectoria suelen estar correlacionadas: no deben contarse como ejecuciones independientes.

2. Bayes y dos tipos de incertidumbre

La regla de Bayes actualiza una creencia previa:

$$p(\theta \mid D) = \frac{p(D \mid \theta) p(\theta)}{p(D)} .$$

En un bandit Bernoulli, una acción da éxito o fracaso. Si su probabilidad desconocida de éxito tiene distribución $\text{Beta}(\alpha, \beta)$, observar un éxito incrementa α y observar un fracaso incrementa β. La media posterior es $\alpha / (\alpha + \beta)$.

Con prior Beta(1,1), tres éxitos y un fracaso producen Beta(4,2), cuya media es 2/3. Esta será la base de Thompson sampling en el capítulo 7.

- **Incertidumbre epistémica:** desconocemos la probabilidad de éxito; los datos pueden reducirla.
- **Incertidumbre aleatoria:** incluso con probabilidad conocida, el siguiente resultado puede seguir siendo impredecible.

Confundir ambas puede hacer que un agente busque ruido en vez de información.

3. Álgebra lineal: tablas, vectores y transformaciones

Debes reconocer un vector, un producto escalar y una multiplicación matriz-vector. Si $w = (2, -1)$ y $\phi(s) = (3, 4)$, entonces $w^\top \phi(s) = 2$. Una función de valor lineal usa precisamente $V_w(s) = w^\top \phi(s)$.

Una matriz de transición contiene probabilidades: cada fila suma 1. Con una política fija, P_π describe cómo cambia el estado. La ecuación de evaluación es

$$V^\pi = r_\pi + \gamma P_\pi V^\pi, \qquad (I - \gamma P_\pi) V^\pi = r_\pi .$$

Para un MDP finito con $\gamma < 1$, podemos resolver este sistema. En código se usa un solucionador lineal; no hace falta calcular explícitamente la inversa.

Ejemplo: un único estado da recompensa 2 cada paso y vuelve a sí mismo. Con $\gamma = 0.9$, $V = 2 + 0.9V$, de donde $V = 20$.

4. Cálculo y optimización

El gradiente indica cómo cambia una función al modificar sus parámetros. Para minimizar una pérdida usamos descenso; para maximizar retorno usamos ascenso:

$$\theta_{\text{nuevo}} = \theta + \eta \nabla_\theta J(\theta) .$$

η es la tasa de aprendizaje: demasiado grande puede causar oscilaciones; demasiado pequeña puede hacer el aprendizaje lento.

La regla de la cadena permite derivar una composición de funciones, como una red neuronal. La identidad más importante para el capítulo 3 es

$$\nabla_\theta p_\theta(x) = p_\theta(x) \nabla_\theta \log p_\theta(x).$$

Permite estimar el gradiente de una recompensa mediante muestras, aunque el entorno no sea diferenciable. Para políticas softmax,

$$\pi_\theta(a) = \frac{e^{\theta_a}}{\sum_b e^{\theta_b}}, \qquad \frac{\partial \log \pi_\theta(a)}{\partial \theta_j} = 1[a = j] - \pi_\theta(j).$$

El laboratorio 3 implementa esta expresión directamente con NumPy. Restar el máximo de los logits antes de exponenciar evita desbordamientos sin cambiar las probabilidades.

5. Aprendizaje automático

Conviene distinguir:

Concepto	Significado	Dónde aparece
Regresión	Predecir una cantidad numérica	Valores, recompensas
Clasificación	Predecir una distribución sobre categorías	Acciones discretas, tokens
Máxima verosimilitud	Aumentar la probabilidad de datos observados	Modelos del mundo, imitación
Sobreajuste	Mejorar en los datos disponibles sin generalizar	Todos los capítulos
Regularización	Restringir o penalizar soluciones	PPO, SAC, RLHF, offline RL
Representación	Transformar observaciones en variables útiles	DQN, Dreamer, JEPA
Cambio de distribución	Evaluar en estados distintos a los de entrenamiento	Modelos aprendidos, offline RL

Una red neuronal es una función parametrizada que transforma entradas en salidas mediante capas. El entrenamiento ajusta parámetros a partir de una pérdida. En RL los datos cambian cuando cambia la política, y los objetivos pueden depender de estimaciones aprendidas. Esto complica el problema respecto a una regresión con datos fijos.

Entrenamiento y evaluación deben separarse. En los laboratorios 2, 3 y 6 calculamos algunas métricas con el modelo conocido del juguete; el agente no utiliza ese conocimiento para actualizarse. Ese acceso es una herramienta de evaluación, no una capacidad que debamos suponer en un problema real.

6. Entropía y divergencia KL

La entropía mide la dispersión de una distribución:

$$H(\pi) = -\sum_a \pi(a) \log \pi(a).$$

Una política determinista tiene entropía 0; una uniforme sobre K acciones tiene entropía $\log K$. Entropía alta significa variedad de acciones, pero no garantiza exploración útil.

La divergencia KL compara distribuciones:

$$D_{\mathrm{KL}}(p\|q) = \sum_a p(a)\log\frac{p(a)}{q(a)}.$$

Es no negativa y, en general, no simétrica. Si q asigna probabilidad cero a un resultado al que p asigna probabilidad positiva, la KL es infinita. En el laboratorio 6 la referencia tiene soporte completo.

Al maximizar $\mathbb{E}_\pi[r] - \beta D_{\mathrm{KL}}(\pi\|\pi_{\mathrm{ref}})$, el primer término favorece respuestas bien puntuadas y el segundo penaliza alejarse de la referencia. Una β grande refuerza esa penalización.

7. Procesos de Markov y notación mínima

La propiedad de Markov afirma que el estado actual contiene la información necesaria para predecir el siguiente estado, dada la acción. No afirma que el mundo sea determinista ni que la observación revele el estado.

Símbolo	Lectura
s_t, o_t	Estado y observación
a_t, r_t	Acción y recompensa de esa transición
$\pi(a \mid s)$	Política: distribución de acciones
$p(s' \mid s, a)$	Modelo de transición
γ	Descuento temporal
G_t	Retorno de una trayectoria desde t
$V^\pi(s)$	Retorno esperado desde s siguiendo π
$Q^\pi(s,a)$	Retorno esperado al empezar con a
$A^\pi(s,a)$	Ventaja: $Q^\pi(s,a) - V^\pi(s)$
α, η	Tasas de aprendizaje
θ, w	Parámetros aprendidos

Usaremos r_t para la recompensa producida por a_t, como en gran parte de Murphy. Otros textos la llaman r_{t+1}; cambia el índice, no el concepto.

8. Python necesario para los laboratorios

Necesitas ejecutar un módulo, importar funciones y trabajar con arrays:

```
import numpy as np
recompensas = np.array([0.0, 0.0, 1.0])
gamma = 0.9
retorno = np.sum(gamma ** np.arange(len(recompensas)) * recompensas)
print(retorno)  # 0.81
```

Un generador aleatorio con semilla permite repetir una ejecución. Mantener la semilla no basta para reproducir cualquier entorno: también importan versiones, código y configuración. Aquí quedan registrados en el manifiesto de resultados.

9. Lo mínimo de LLM y teoría de juegos

Para el capítulo 5, piensa en una matriz: las filas son las acciones de un jugador, las columnas las del otro y cada casilla indica las recompensas. Una estrategia mixta es una distribución sobre acciones. Una mejor respuesta maximiza la recompensa frente a una estrategia rival fija.

Para el capítulo 6, un token es una unidad de texto. Un modelo autorregresivo asigna

$$p(y \mid x) = \prod_{t=1}^{T} p(y_t \mid x, y_{<t}).$$

Una respuesta completa es una acción grande; cada token también puede verse como una acción dentro de un episodio. SFT aprende de respuestas demostradas; RLFT aprende a partir de recompensas. Para la primera lectura no necesitas implementar atención, LoRA ni entrenamiento distribuido.

10. Autodiagnóstico con soluciones

Intenta resolver estas preguntas antes de mirar las respuestas.

1. ¿Cuánto vale una secuencia de recompensas $(1, 0, 2)$ con $\gamma = 0.5$?
2. ¿Un valor esperado de 3 implica observar recompensa 3?
3. ¿Qué posterior resulta de Beta(1,1), dos éxitos y tres fracasos?
4. Si $Q(s, a) = 4$ y $V(s) = 3$, ¿cuál es la ventaja?
5. ¿Qué diferencia hay entre observación y estado?
6. ¿Qué hace el signo positivo en el ascenso por gradiente?
7. ¿Una política de alta entropía siempre es mejor?
8. ¿Podemos concluir que A supera a B porque su mejor semilla obtuvo más recompensa?

Soluciones: 1) $1 + 0.5^2\,(2) = 1.5$. 2) No: es una media ponderada. 3) Beta(3,4), media 3/7. 4) 1. 5) La observación puede ocultar información relevante del estado. 6) Intenta aumentar el objetivo localmente. 7) No: depende de recompensa, incertidumbre y objetivo. 8) No:

compara distribuciones de ejecuciones bajo un protocolo común.

Si fallas 1–4, dedica una sesión a probabilidad y notación. Si fallas 5–8, empieza con los laboratorios 1 y 8 antes de interpretar curvas de aprendizaje.

Ruta de preparación

Una propuesta flexible: 60–90 minutos para probabilidad y Bayes, 45–60 para álgebra y cálculo, 45 para ML y entropía, y 30–45 para ejecutar el primer cuaderno. Son tiempos de estudio orientativos, no requisitos del libro.

Fuente y alcance: material preparatorio elaborado para esta guía, conectado con Murphy v5, capítulos 1–3. Las explicaciones y ejercicios son propios.

Capítulo 1 · Introducción

Lectura: pp. 13–30 del PDF; §§1.1–1.3. Capítulo original.

Idea central: antes de elegir un algoritmo, hay que especificar qué observa el agente, qué puede hacer, qué intenta optimizar y qué desconoce.

Resumen

En §1.1 Murphy presenta RL como parte de la toma de decisiones secuencial. El agente recibe observaciones, mantiene un estado interno y elige acciones mediante una política. Sus decisiones afectan a lo que verá y podrá hacer después. La información disponible, el estado real del mundo y los parámetros del agente son objetos distintos: aprender una política y actualizar una creencia no son la misma operación.

El objetivo habitual es maximizar retorno esperado. En tareas episódicas la interacción termina; en tareas continuas puede usarse un objetivo descontado o de recompensa media. El descuento controla cuánto pesan las consecuencias lejanas y, con recompensas acotadas y $\gamma < 1$, hace finita la suma infinita. Cambiarlo puede cambiar la política óptima.

En §1.2 se ordenan varios modelos. Un MDP supone que el estado relevante es observado. Un POMDP permite observaciones parciales y necesita memoria o creencias sobre el estado oculto. Un estado de creencia resume probabilísticamente el historial cuando el modelo es conocido. También podemos mantener creencias sobre parámetros desconocidos: esto conecta la decisión con el aprendizaje del entorno.

Un bandit contextual tiene una decisión por contexto sin que la acción determine los contextos futuros; aun así, la acción determina qué recompensa observamos y qué aprendemos. Los MDP condicionados por metas permiten modificar el objetivo; los MDP contextuales permiten variar la tarea o dinámica entre episodios. La identificación del mejor brazo, la optimización bayesiana, el aprendizaje activo y hasta la elección de pasos de SGD se presentan como problemas relacionados de decisión e información.

En §1.3 aparece el mapa del resto del libro: aprender valores, aprender directamente políticas o aprender modelos para planificar. Esta clasificación es independiente de usar tablas o redes neuronales, y de aprender on-policy u off-policy. La observación parcial exige memoria; el desconocimiento del modelo exige exploración. Son problemas diferentes.

La recompensa tampoco es un detalle secundario. Una señal escasa dificulta aprender; una señal incorrecta puede producir un comportamiento eficaz para la puntuación e indeseable para la tarea. El shaping añade información, pero puede modificar el objetivo. El shaping basado en potencial tiene garantías bajo condiciones apropiadas. El capítulo termina con recomendaciones sobre variabilidad, agregación y evaluación experimental.

Ecuaciones que debes poder explicar

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k}, \qquad J(\pi) = \mathbb{E}_\pi[G_0].$$

G_t corresponde a una trayectoria; J promedia sobre las trayectorias que producen política y entorno.

Para shaping basado en potencial:

$$r'_t = r_t + \gamma\Phi(s_{t+1}) - \Phi(s_t).$$

La suma adicional se telescopa. Con potencial acotado en horizonte infinito descontado, o potencial terminal cero en episodios, añade un término independiente de las acciones desde un estado inicial fijo. Sin cuidar los estados terminales esa conclusión puede fallar.

Experimento 1 · Futuro y memoria

Pregunta: ¿cambia la mejor decisión cuando modificamos el descuento o la información disponible?

Hipótesis: recibir 3 en $t = 4$ supera recibir 1 ahora únicamente si $3\gamma^4 > 1$. Por otra parte, recordar una señal inicial permite resolver una tarea cuya observación final es ambigua.

El primer panel compara esos dos retornos para 201 valores de γ. El segundo genera un bit oculto equiprobable, lo revela al inicio y lo oculta antes de decidir. Compara una decisión aleatoria sin memoria con una política que recuerda la señal. Son políticas prescritas: este laboratorio ilustra la formulación, no entrena un agente.

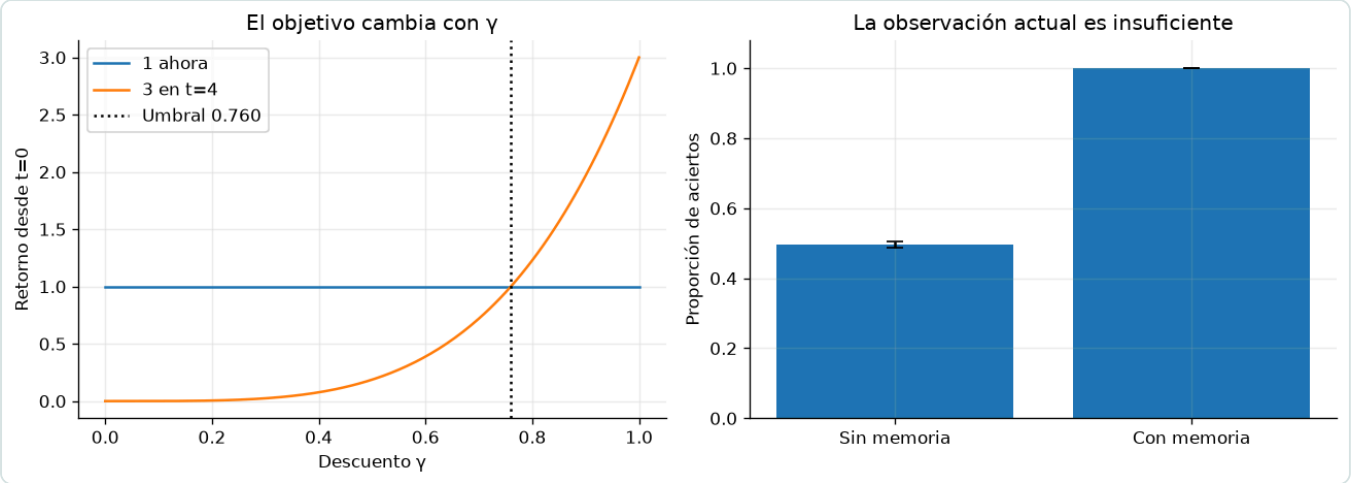
```
uv run python -m experimentos.ejecutar --capitulo 1
```

Qué medir: punto de cruce de retornos y proporción de aciertos. Se realizan 20 réplicas de 1.000 episodios para la tarea de memoria.

Qué observar: el cruce está en $\gamma = (1/3)^{1/4} \approx 0.760$; la política sin memoria ronda 50% y la que recuerda alcanza 100% en este entorno sin ruido. La igualdad en el umbral significa empate.

Modificación: cambia el retraso de cuatro a ocho pasos. Después introduce ruido en la señal inicial y comprueba que recordar ya no basta para acertar siempre.

Límite: no demuestra que una red recurrente vaya a aprender esa memoria. Solo demuestra que la información necesaria no está en la última observación.



Para el grupo

1. ¿Qué habría que incluir en el estado de un robot para que la representación sea Markoviana?
2. ¿Puede un bandit necesitar exploración aunque sus acciones no cambien el mundo?
3. ¿Cómo distinguirías un fallo de aprendizaje de un fallo de recompensa?

Conexión: el capítulo 2 aprende a valorar consecuencias; el 3 aprende a actuar; el 4 aprende a predecirlas.

Capítulo 2 · Métodos basados en valores

Lectura: pp. 31–50; §§2.1–2.5. Capítulo original.

Idea central: podemos aprender cuánto conviene cada acción y elegir las de mayor valor, propagando información mediante las ecuaciones de Bellman.

Resumen

En §2.1 se distinguen el valor de un estado, el valor de una acción y su ventaja respecto a la política. Las ecuaciones de Bellman descomponen un problema de largo plazo en recompensa inmediata y valor del siguiente estado. Evaluar una política fija usa un promedio sobre sus acciones; buscar la política óptima introduce un máximo.

Con modelo conocido, §2.2 presenta iteración de valor, iteración de políticas y RTDP. La primera aplica repetidamente el operador de optimalidad; la segunda alterna evaluación y mejora. RTDP concentra el trabajo en estados relevantes para el problema. En el caso finito descontado, la contracción de Bellman da una razón precisa para la convergencia. El espacio de estados puede seguir siendo demasiado grande para calcularlo todo.

Cuando solo tenemos muestras, §2.3 compara Monte Carlo y diferencias temporales. Monte Carlo espera al retorno observado; TD usa una estimación del futuro como parte de su objetivo, operación llamada bootstrap. TD puede actualizar antes y reducir varianza, pero incorpora errores de su estimación. Los retornos de varios pasos, TD(λ) y las trazas de elegibilidad distribuyen crédito entre distintos horizontes.

SARSA (§2.4) actualiza usando la siguiente acción realmente elegida por la política de comportamiento. Q-learning (§2.5) usa el máximo estimado sobre acciones siguientes. Esta diferencia explica su carácter on-policy y off-policy, respectivamente: Q-learning puede aprender sobre una política greedy mientras recoge datos exploratorios.

La aproximación funcional permite reemplazar tablas enormes por redes. DQN combina aprendizaje de Q con replay y una red objetivo. Reutilizar experiencias mejora el aprovechamiento de datos; una red objetivo cambia más despacio que la red entrenada. Estas técnicas son útiles frente a la combinación problemática de aproximación funcional, bootstrap y aprendizaje off-policy, la llamada tríada mortal. Las garantías tabulares no se trasladan automáticamente al caso neuronal.

El máximo de estimaciones ruidosas introduce sobreestimación. Double Q separa selección y evaluación; Double DQN traslada la idea al caso neuronal. El capítulo también recorre replay priorizado, arquitecturas dueling, noisy nets, retornos de varios pasos, Rainbow y BBF. Son combinaciones de decisiones estadísticas y de ingeniería, no sustitutos del principio de Bellman. Finalmente, HER reutiliza trayectorias fallidas como ejemplos de metas que sí se alcanzaron, recalculando las recompensas correspondientes.

Ecuaciones esenciales

$$Q^{\pi}(s,a) = \mathbb{E}[r_t + \gamma V^{\pi}(s_{t+1}) \mid s_t = s, a_t = a], \qquad A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s).$$

Q-learning tabular:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma(1-d)\max_b Q(s',b) - Q(s,a)].$$

$d = 1$ indica un terminal real. En SARSA el máximo se reemplaza por $Q(s',a')$, con $a' \sim \pi(\cdot \mid s')$. Un límite artificial de tiempo no equivale necesariamente a un terminal del problema: hay que definir qué retorno queremos estimar.

Experimento 2 · Aprender Bellman mediante muestras

Pregunta: ¿puede Q-learning aproximarse al resultado de un planificador que conoce el modelo?

Usamos una cadena con siete estados: 0 y 6 son terminales, se empieza cada episodio en un estado interior aleatorio y se evalúa desde el estado 3. Hay acciones izquierda/derecha y una probabilidad de 0.15 de ejecutar la dirección contraria. Entrar en 6 da recompensa 1; las demás transiciones no terminales de origen dan −0.01. Después de terminar no se reciben más recompensas.

El planificador calcula Q^* con el modelo exacto. El agente aprende únicamente de transiciones, con $\epsilon = 0.25$, $\gamma = 0.95$ y tasa por visita $N(s,a)^{-0.6}$. Se comparan 20 ejecuciones de 16.000 transiciones reales.

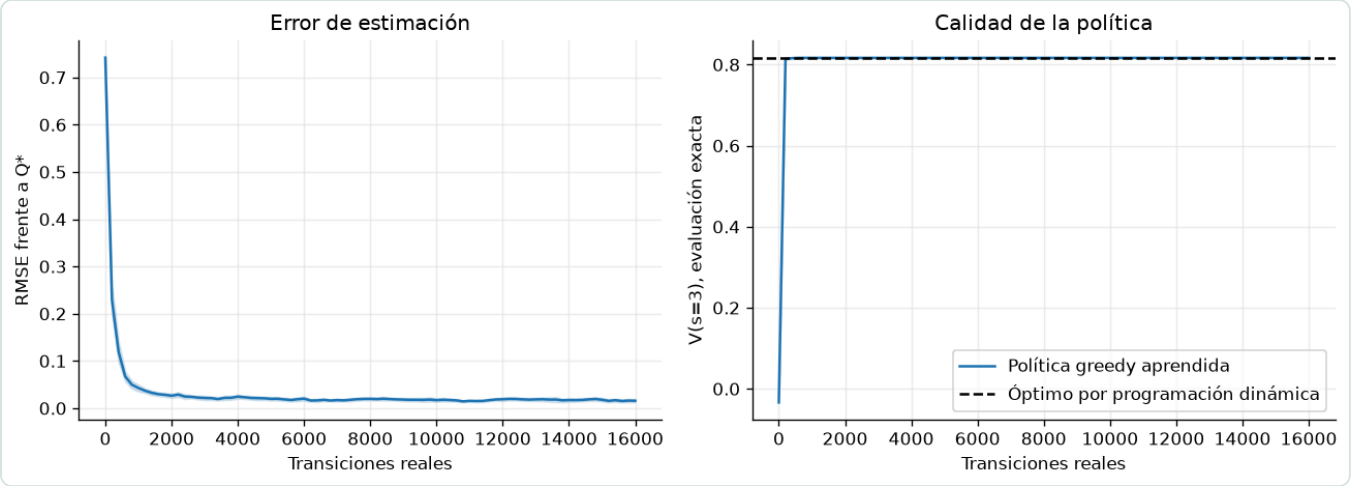
```
uv run python -m experimentos.ejecutar --capitulo 2
```

Métricas: RMSE entre Q y Q^* sobre pares no terminales, y valor exacto de la política greedy aprendida desde el estado 3. El evaluador conoce el modelo; el aprendiz no lo consulta.

Qué observar: el error disminuye y la política puede llegar al valor óptimo antes de que Q sea muy preciso. Para elegir bien basta con ordenar correctamente las acciones relevantes. Los resultados guardados alcanzan el valor óptimo aproximado 0.8166, con error de Q pequeño pero no nulo.

Modificación: compara varias probabilidades de deslizamiento o reduce la exploración. Una única ejecución no permite concluir que un cambio de hiperparámetro sea mejor.

Límite: este ejemplo es tabular; no prueba estabilidad de DQN ni reproduce la tríada mortal. Tampoco justifica trasladar su tasa de aprendizaje sin cambios a una red.



Para el grupo

1. ¿Por qué SARSA puede preferir una ruta segura mientras sigue explorando?
2. ¿Cuál es la diferencia entre error de Q y pérdida de rendimiento de la política?
3. ¿Por qué una red objetivo ayuda pero no ofrece por sí sola una garantía universal?

Conexión: el capítulo 3 usará estimaciones de valor para mejorar directamente una política parametrizada.

Capítulo 3 · Métodos basados en políticas

Lectura: pp. 51–82; §§3.1–3.6. Capítulo original.

Idea central: ajustar directamente la distribución de acciones permite optimizar políticas estocásticas y manejar acciones continuas, pero exige controlar la varianza y el tamaño de las actualizaciones.

Resumen

En §3.1 se deriva el gradiente mediante la identidad de la derivada logarítmica. No hace falta diferenciar la dinámica: basta muestrear trayectorias y conocer la probabilidad de las acciones elegidas. REINFORCE aumenta la probabilidad de acciones asociadas a mejores retornos. Usar recompensa futura en vez del retorno completo evita atribuir a una acción recompensas que ocurrieron antes.

Restar un baseline que no dependa de la acción, tratado como constante en el estimador, conserva la esperanza del gradiente. Un baseline adecuado puede reducir mucho su varianza. El valor del estado es una elección habitual; no cualquier baseline reduce la varianza.

En §3.2, actor-critic combina una política con un estimador de valor. El crítico permite usar bootstrap para actualizar sin esperar al episodio completo. A2C usa un error TD como estimación de ventaja. GAE mezcla errores TD a distintos horizontes mediante λ, intercambiando sesgo y varianza. La calidad del crítico y la coordinación de las dos tasas de aprendizaje importan.

El gradiente natural tiene en cuenta cómo los cambios de parámetros modifican distribuciones, usando la geometría de Fisher. Para acciones continuas, DDPG entrena un actor determinista guiado por un crítico. TD3 combate errores y sobreestimación mediante dos críticos, actualizaciones retardadas y suavizado del objetivo. WPO explora otra geometría para mover distribuciones de acciones.

En §3.3, TRPO y PPO buscan limitar cambios destructivos. TRPO aproxima una actualización restringida por KL. PPO usa un objetivo recortado más sencillo y permite varias actualizaciones sobre un lote. El clipping limita incentivos en ese objetivo; no impone por sí solo una cota dura a toda la divergencia ni garantiza mejora en cada ejecución. VMPO ofrece una alternativa basada en distribución auxiliar y proyección.

§3.4 explica cómo reutilizar datos de otra política. El muestreo por importancia corrige el cambio de distribución cuando hay soporte suficiente, pero sus productos pueden tener mucha varianza. V-trace y Retrace introducen correcciones y truncamientos. IMPALA combina estos principios con recogida distribuida. Off-policy no significa automáticamente que un conjunto fijo y arbitrario de datos sea suficiente.

§3.5 recuerda que también existen búsqueda aleatoria, estrategias evolutivas y otros métodos sin gradientes. Finalmente, §3.6 interpreta control como inferencia: favorecer trayectorias de alta recompensa conduce a objetivos con entropía o KL. MPO, AWR/AWAC y SAC se entienden desde esa perspectiva. SAC aprende una política estocástica con un objetivo de recompensa más entropía. La discusión de inferencia activa conecta percepción, preferencias y búsqueda de información.

Ecuaciones esenciales

Para episodios sin descuento, una forma de REINFORCE es

$$\nabla_\theta J = \mathbb{E}[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)(G_t - b(s_t))].$$

Si $J = \mathbb{E}[\sum_t \gamma^t r_t]$ y G_t descuenta desde t, la forma por trayectoria incluye además un factor γ^t. Otras presentaciones lo incorporan a la distribución de visitas. En el laboratorio hay un solo paso y no surge esta distinción.

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t), \qquad \hat{A}_t^{\mathrm{GAE}} = \sum_{l \geq 0} (\gamma\lambda)^l \delta_{t+l}.$$

La suma se trunca y los valores terminales se tratan adecuadamente en episodios finitos.

Experimento 3 · ¿Qué aporta un baseline?

Un bandit contextual tiene dos contextos equiprobables y tres acciones. Las medias, sin un desplazamiento común, son $(1, 0, -1)$ y $(-1, 0, 1)$. Añadimos 5 a todas las recompensas y ruido normal con desviación 0.5.

Se entrenan dos políticas softmax con REINFORCE: una sin baseline y otra con una media móvil por contexto, calculada con lotes anteriores. Usamos 500 actualizaciones, lotes de 32 y 20 semillas.

```
uv run python -m experimentos.ejecutar --capitulo 3
```

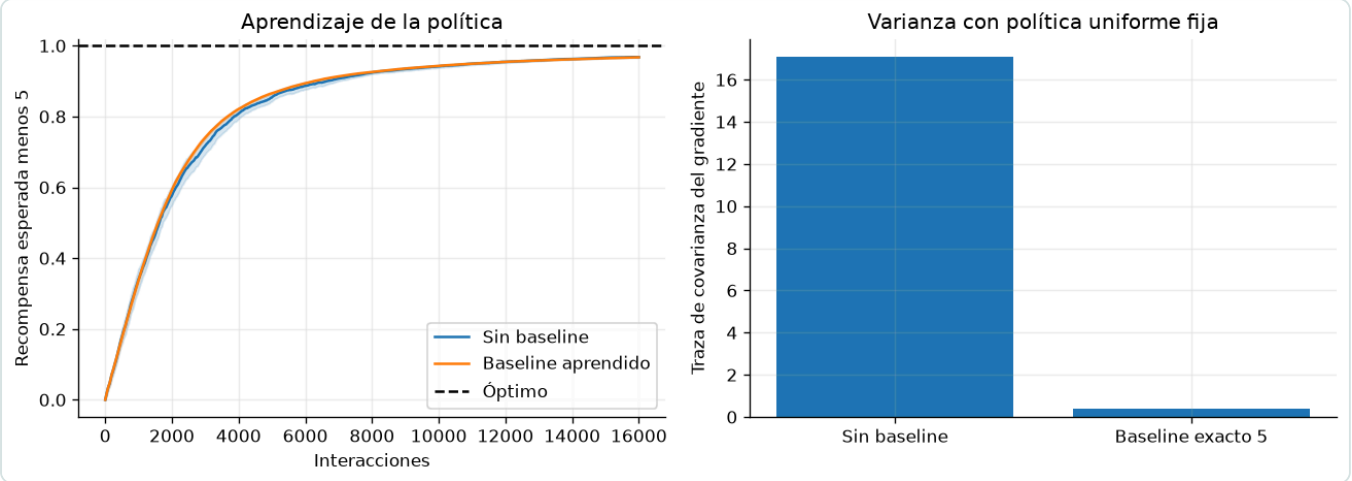
Control importante: además del aprendizaje, medimos ambos estimadores bajo una misma política uniforme fija, con 50.000 muestras y baseline exacto 5. Comparar varianzas de políticas distintas no aislaría el efecto del baseline.

Métricas: recompensa esperada exacta menos el desplazamiento 5; y traza de la covarianza del gradiente en el control fijo.

Resultado observado: la varianza cae aproximadamente de 17.07 a 0.392. Ambas políticas alcanzan recompensas finales parecidas, alrededor de 0.967 frente al óptimo 1. Reducir varianza no implica necesariamente obtener una media final superior con este presupuesto.

Modificación: elimina el desplazamiento 5 y repite. Luego usa deliberadamente un baseline muy inadecuado. Razona por qué conservar la esperanza no garantiza reducir la varianza.

Límite: ilustra REINFORCE y baselines; no implementa PPO, GAE ni un crítico neuronal.



Para el grupo

1. ¿Por qué un baseline dependiente de la acción necesita tratamiento adicional?
2. ¿Qué diferencia hay entre menos varianza y menos sesgo?
3. ¿Qué permite afirmar el clipping de PPO y qué habría que medir aparte?

Conexión: las ideas de gradiente, ventaja y KL reaparecen en el ajuste de LLM del capítulo 6.

Capítulo 4 · Métodos basados en modelos

Lectura: pp. 83–120; §§4.1–4.5. Capítulo original.

Idea central: aprender cómo funciona el entorno permite convertir cómputo en experiencia imaginada; la utilidad de esa experiencia depende de la fidelidad del modelo.

Resumen

§4.1 plantea el intercambio fundamental: una interacción real puede ser costosa, mientras que simularla puede ser barato. Aprender un modelo de transiciones y recompensas permite buscar acciones o entrenar políticas dentro del modelo. Sin embargo, una política puede descubrir y explotar errores de predicción. Por eso la recogida de datos, el ajuste del modelo y la mejora de la política suelen alternarse.

§4.2 estudia la planificación en el momento de decidir. La búsqueda hacia delante considera futuros posibles; branch and bound y métodos heurísticos reducen la exploración del árbol. MCTS asigna simulaciones de forma selectiva, combinando estimaciones de valor y exploración. AlphaZero usa reglas conocidas; MuZero aprende representaciones y una dinámica orientada a recompensas, valores y políticas.

El control predictivo, MPC, optimiza una secuencia de acciones, ejecuta la primera y vuelve a planificar tras observar el nuevo estado. La secuencia interna es de lazo abierto, aunque el controlador completo incorpora realimentación al replanificar. En entornos estocásticos, una política contingente puede superar a cualquier secuencia fija porque anticipa cómo aprovechar futuras observaciones. LQR, CEM, random shooting, MPPI y métodos con procesos gaussianos ofrecen distintas formas de resolver o aproximar la optimización.

§4.3 cambia el momento de usar el modelo: la planificación en segundo plano produce datos imaginados que mejoran una política reactiva. Dyna combina actualizaciones a partir de datos reales y simulados. La perspectiva de juegos y optimización a dos niveles ayuda a pensar cómo coordinar el modelo y la política. Aquí “planificación offline” significa fuera del instante de decisión; no implica necesariamente el conjunto fijo de datos de offline RL del capítulo 7.

§4.4 pregunta qué debería predecir el modelo. Reconstruir observaciones completas proporciona una señal rica, pero puede gastar capacidad en detalles irrelevantes. Dreamer aprende una representación dinámica y entrena en imaginación; IRIS usa representaciones discretas y modelos autorregresivos. Otras familias predicen recompensas, valores, políticas o representaciones futuras, buscando preservar información útil para decidir.

Aprender representaciones por autopredicción introduce el riesgo de colapso: mapear todas las observaciones al mismo vector hace la predicción fácil e inútil. Objetivos auxiliares, redes objetivo y regularizadores intentan evitarlo; sus garantías dependen del

caso. JEPA, BYOL y TD-MPC muestran distintas combinaciones. El capítulo también aborda error acumulado, rollouts cortos, incertidumbre y la posibilidad de alternar métodos con y sin modelo.

§4.5 estudia predicciones a más de un paso. Las funciones de valor generalizadas predicen señales distintas de la recompensa. Las representaciones y características sucesoras acumulan ocupaciones o características futuras bajo una política. Así puede reevaluarse rápidamente una recompensa nueva, siempre respetando la dependencia respecto a la política y la representación. La mejora generalizada de políticas reutiliza una biblioteca de habilidades; las medidas sucesoras conectan predicción y muestreo de futuros.

Ecuación esencial

Si la recompensa es lineal en características de transición,

$$r(s,a,s') = \phi(s,a,s')^\top w, \qquad \psi^\pi(s,a) = \mathbb{E}_\pi\left[\sum_{t\geq 0} \gamma^t \phi(s_t,a_t,s_{t+1}) \mid s_0 = s, a_0 = a\right],$$

entonces

$$Q_w^\pi(s,a) = \psi^\pi(s,a)^\top w\,.$$

Cambiar w permite evaluar otra recompensa sin reaprender ψ^π. Si cambia la dinámica o queremos una política distinta, la reutilización necesita trabajo adicional.

Experimento 4 · Dyna-Q

Pregunta: ¿cuánto se gana en aprendizaje por interacción real cuando reutilizamos un modelo aprendido?

Una cadena determinista tiene nueve estados; los extremos son terminales. El agente empieza en 1 y recibe 1 al llegar a 8. Las demás recompensas son cero. Se comparan 0, 5 y 20 actualizaciones imaginadas por transición real, con el mismo presupuesto de 1.600 transiciones y 20 semillas.

Cada transición observada actualiza Q y una tabla del modelo. La planificación muestrea solo pares estado-acción ya observados y aplica otra actualización de Bellman. No se entrega el modelo real al aprendiz. En este entorno determinista, almacenar una transición basta para conocer ese par.

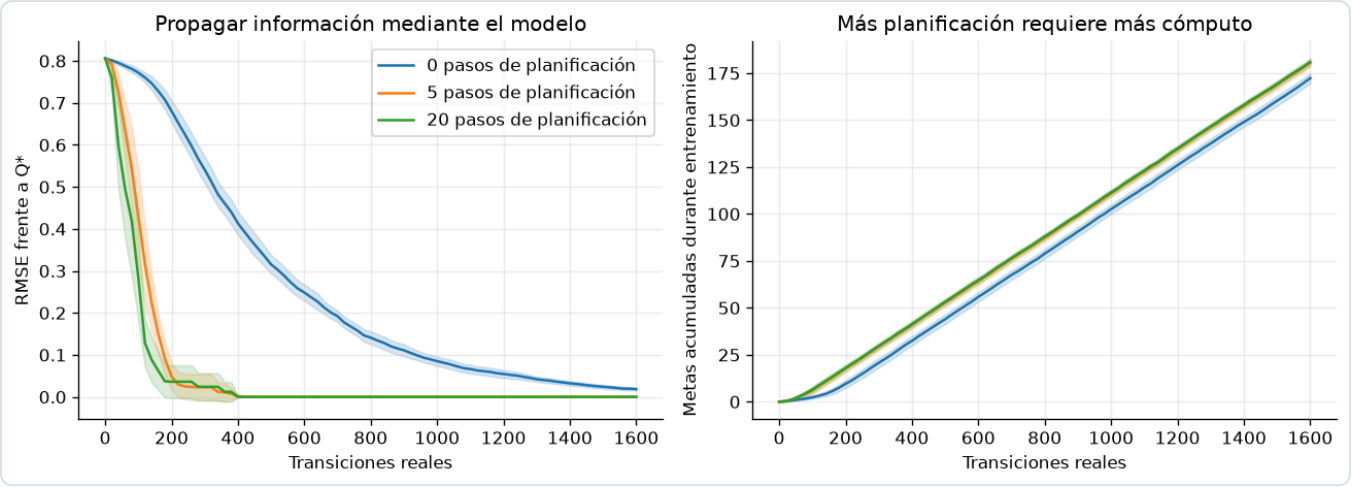
```
uv run python -m experimentos.ejecutar --capitulo 4
```

Métricas: error frente a Q^* y metas alcanzadas durante el entrenamiento. También se registran las actualizaciones totales: 1.600, 9.600 y 33.600.

Qué observar: la planificación propaga antes la información de la meta y reduce el error con menos datos reales. En los resultados guardados, 5 y 20 pasos alcanzan prácticamente el mismo error final; veinte pasos no aportan una mejora proporcional a su coste.

Modificación: compara a igual número total de actualizaciones, no solo a igual cantidad de datos. Como extensión de implementación, añade dinámica estocástica: una tabla que recuerde únicamente la última transición ya no será un modelo fiel de su distribución.

Límite: aquí el modelo observado es exacto y no existe error acumulado de una red. El experimento no demuestra que planificar más siempre sea mejor ni reproduce Dreamer o MuZero.



Para el grupo

1. ¿Cuándo vale más una transición real que miles de transiciones imaginadas?
2. ¿Por qué predecir bien los píxeles puede ser insuficiente para actuar bien?
3. ¿Qué diferencia hay entre cambiar la recompensa y cambiar la dinámica al reutilizar características sucesoras?

Conexión: aprender un modelo y una política a la vez anticipa los problemas de interacción y no estacionariedad del capítulo 5.

Capítulo 5 · Aprendizaje por refuerzo multiagente

Lectura: pp. 121–146; §§5.1–5.3. Capítulo original.

Idea central: cuando otros agentes también aprenden, “hacer lo mejor” depende de sus estrategias y del concepto de solución que se adopte.

Resumen

§5.1 amplía el modelo del entorno. Los juegos de forma normal describen decisiones conjuntas; los juegos estocásticos incorporan estados y transiciones; los POSG incluyen observaciones parciales. Las recompensas pueden ser compartidas, de suma cero o generales. En juegos extensivos importan el orden de las decisiones y los conjuntos de información: dos historias pueden ser indistinguibles para un jugador.

Desde la perspectiva de un agente, cambiar la política de otro cambia el entorno efectivo, aunque las reglas permanezcan fijas. Esta no estacionariedad complica aplicar algoritmos diseñados para un único agente. La información pública y privada también condiciona qué puede aprenderse y qué puede utilizarse al actuar.

§5.2 distingue conceptos que no deben confundirse. Una mejor respuesta es óptima frente a rivales fijos. En suma cero de dos jugadores, minimax busca robustez frente al adversario. Un equilibrio de Nash no permite mejora unilateral, pero puede ser socialmente malo o no ser único. El equilibrio correlacionado permite recomendaciones dependientes de una señal común. Pareto, bienestar y equidad expresan criterios diferentes.

La explotabilidad y NashConv miden vulnerabilidad frente a mejores respuestas. Una recompensa media de cero no basta para certificar equilibrio: una estrategia puede empatar contra un rival particular y ser muy explotable por otro. El arrepentimiento compara resultados con acciones o políticas alternativas. Los valores de Shapley estudian contribuciones individuales a resultados compartidos. Stackelberg modela compromiso y respuesta entre líder y seguidor.

§5.3 recorre estrategias de aprendizaje. Centralizar todas las decisiones es sencillo conceptualmente, pero puede ser costoso o incompatible con información local. Aprender de forma independiente es práctico, aunque puede oscilar. CTDE permite críticos con información global durante entrenamiento y políticas locales durante ejecución. VDN y QMIX descomponen valores conjuntos bajo restricciones que facilitan escoger acciones por separado.

El self-play aprovecha la simetría, pero no ofrece convergencia universal. Fictitious play responde a la estrategia media del rival; las poblaciones y PSRO amplían el conjunto de oponentes para mejorar robustez. CFR minimiza arrepentimiento contrafactual en conjuntos de información; sus garantías clásicas se refieren a políticas medias y juegos bajo supuestos concretos, como dos jugadores de suma cero y recuerdo perfecto.

El capítulo cierra con métodos regularizados como MMD, comparaciones con PPO, planificación al decidir y aplicaciones a agentes lingüísticos. Es crucial separar resultados demostrados en juegos tabulares de observaciones empíricas con aproximación neuronal.

Ecuaciones esenciales

Un equilibrio de Nash satisface, para cada jugador i,

$$J_i(\pi_i, \pi_{-i}) \geq J_i(\pi_i', \pi_{-i}) \quad \text{para cualquier } \pi_i' .$$

En un juego matricial de suma cero con matriz de pagos A, el laboratorio usa

$$\text{NashConv}(p, q) = \max_i (Aq)_i - \min_j (p^\top A)_j .$$

Es la suma de las dos mejoras unilaterales posibles. Algunas convenciones de explotabilidad dividen esta cantidad entre dos; aquí no lo hacemos.

Experimento 5 · Piedra, papel y tijera

Pregunta: ¿la estrategia actual y la estrategia media se comportan igual durante el aprendizaje?

Dos jugadores usan regret matching durante 4.000 iteraciones. Mantienen el arrepentimiento acumulado de cada acción y asignan probabilidad proporcional a su parte positiva. Se inicializan con distribuciones aleatorias para evitar el caso trivial de empezar exactamente en el equilibrio uniforme.

Calculamos los pagos esperados de todas las acciones frente al rival. Es un experimento de información completa, no un aprendizaje basado únicamente en el premio de una acción muestreada.

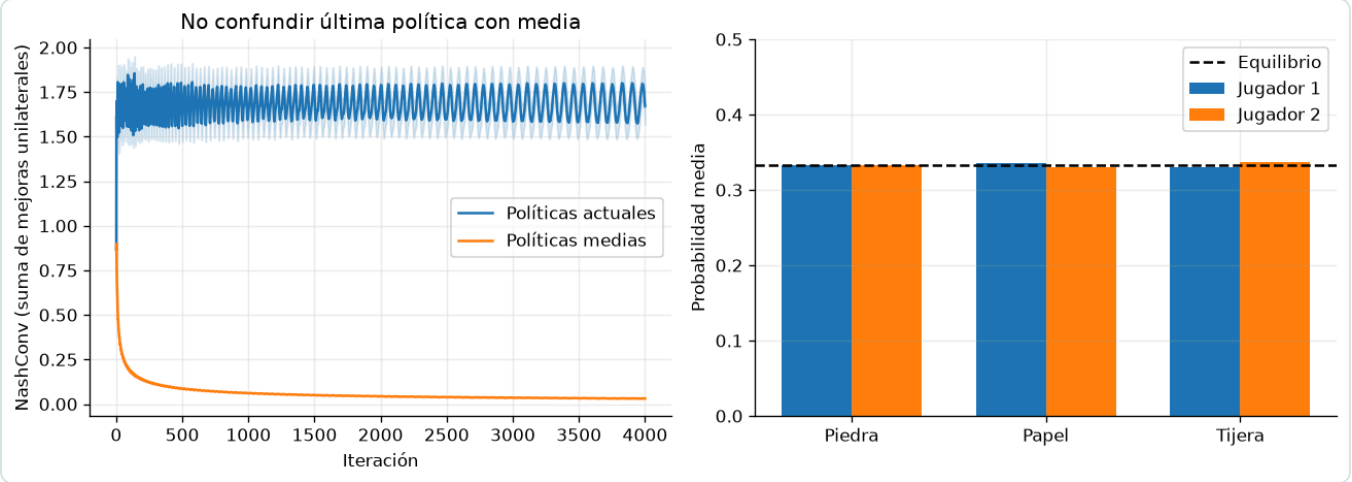
```
uv run python -m experimentos.ejecutar --capitulo 5
```

Métricas: NashConv de las políticas actuales y de las medias temporales; distribución media sobre las tres acciones. Las 20 réplicas corresponden a inicializaciones diferentes.

Resultado observado: la política media se aproxima a $(1/3, 1/3, 1/3)$ y su NashConv final ronda 0.031. La de las políticas actuales ronda 1.67: no se estabilizan del mismo modo. La media entre semillas que se dibuja en una curva no sustituye a la media temporal que usa el algoritmo.

Modificación: enfrenta la política aprendida a un rival que siempre juega piedra. Una mejor respuesta explotará su debilidad, mientras la estrategia de equilibrio prioriza robustez ante cualquier rival.

Límite: implementamos regret matching en un juego matricial, un caso elemental relacionado con CFR. No implementamos CFR para póquer ni demostramos resultados para cooperación o suma general.



Para el grupo

1. ¿Por qué “nadie quiere desviarse” no equivale a “todos están satisfechos”?
2. ¿Qué información puede usar un crítico centralizado que no debería recibir el actor al ejecutar?
3. ¿Qué conclusiones cambiarían si solo observáramos pagos de acciones elegidas?

Conexión: no estacionariedad, preferencias y self-play reaparecen al entrenar sistemas con LLM.

Capítulo 6 · LLM y aprendizaje por refuerzo

Lectura: pp. 147–172; §§6.1–6.4. Capítulo original.

Idea central: RL puede mejorar un modelo de lenguaje, y un modelo de lenguaje puede aportar representaciones, recompensas, modelos o políticas a un sistema de RL. Son dos direcciones diferentes.

Resumen

§6.2 distingue preentrenamiento, ajuste supervisado y ajuste con recompensas. SFT aprende de respuestas demostradas; RLFT favorece respuestas que obtiene buenas puntuaciones al generarlas. Verificar a veces es más sencillo que producir una solución. RLVR usa señales verificables; RLHF aprende recompensas a partir de preferencias humanas; RLAIF usa evaluaciones de otros modelos. Las recompensas de resultado evalúan el final; las de proceso ofrecen información intermedia, pero pueden ser más difíciles de especificar o aprender.

El capítulo conecta la generación de tokens intermedios con cómputo adicional antes de responder. Discute qué capacidades aporta el modelo base y cuáles puede potenciar RL; no conviene interpretar automáticamente mejoras como aparición de capacidades desde cero. También separa un sistema de módulos con un objetivo común de un juego donde varios agentes persiguen objetivos propios.

En una interacción de un solo turno, el prompt puede ser contexto y la respuesta completa una acción: un bandit contextual. Si desplegamos la generación token a token, el estado es el prefijo. PPO, GRPO, DAPO, GSPO, RLOO y REINFORCE++ difieren en cómo estiman ventajas, normalizan, recortan razones de probabilidad o agregan pérdidas. GRPO estima una referencia con respuestas del mismo prompt; RLOO excluye la recompensa de la propia muestra al calcular su baseline. Esto permite prescindir de un crítico, pero requiere suficientes muestras y puede aumentar el coste.

La regularización KL intenta evitar cambios excesivos respecto al modelo de referencia. DPO conecta la solución del problema regularizado con un modelo de preferencias y optimiza pares preferido/rechazado directamente. Es una formulación distinta de un bucle online de política y recompensa. El muestreo guiado en inferencia y el ajuste con RL también pueden entenderse como maneras de aproximar una distribución favorecida por la recompensa, pagando el cómputo en momentos diferentes.

En problemas de varios turnos, acciones como herramientas y respuestas afectan a un entorno externo. Surgen consecuencias retrasadas, estados ocultos, acciones inválidas y contextos crecientes. El retorno y el crédito deben asignarse a través de la interacción. RAGEN, entrenamiento por turnos y métodos con críticos abordan partes del problema. La discusión de alineamiento y juegos de asistencia destaca que la utilidad humana puede ser incierta; optimizar un proxy fijo no elimina esa incertidumbre.

§6.3 invierte la dirección: usa LLM para interpretar observaciones, proponer recompensas, generar modelos del mundo, producir acciones o escribir código que implemente modelos y políticas. Generar un programa reutilizable puede ser mucho más rápido que llamar al modelo en cada paso, pero su corrección debe comprobarse. La latencia y el tamaño del contexto importan; se mencionan alternativas recurrentes y arquitecturas híbridas.

§6.4 muestra un ciclo de entrenamiento con Tinker y GEM: recoger episodios, calcular retornos y ventajas, preparar pérdidas por token, actualizar y registrar estabilidad. Los tokens del entorno no son decisiones del agente y se enmascaran. El ejemplo del libro usa servicios de entrenamiento; nuestro laboratorio es una reducción local y no reproduce esa infraestructura.

Ecuaciones esenciales

Para una respuesta y a un prompt x,

$$J(\pi) = \mathbb{E}_{y\sim\pi(\cdot\mid x)}[r(x,y)] - \beta D_{\mathrm{KL}}(\pi(\cdot\mid x)\|\pi_{\mathrm{ref}}(\cdot\mid x)).$$

Si $\beta > 0$, hay soporte apropiado y podemos representar cualquier distribución, la solución es

$$\pi^*(y\mid x) = \frac{\pi_{\mathrm{ref}}(y\mid x)\exp(r(x,y)/\beta)}{\sum_{y'}\pi_{\mathrm{ref}}(y'\mid x)\exp(r(x,y')/\beta)}.$$

Para un grupo de $K > 1$ respuestas independientes, RLOO usa $b_i = \frac{1}{K-1}\sum_{j\neq i} r_j$. La recompensa de la propia muestra no entra en su baseline.

Experimento 6 · Optimizar el proxy equivocado

Pregunta: ¿puede mejorar la recompensa mientras empeora la respuesta real?

Un único prompt tiene cuatro respuestas posibles:

Respuesta	Probabilidad inicial	Recompensa real	Recompensa proxy
Correcta breve	0.5	1	1
Correcta larga	0.2	1	1.2
Incorrecta convincente	0.1	0	1.8
Irrelevante	0.2	0	−0.2

Entrenamos logits categóricos con RLOO y penalización KL. Comparamos cuatro coeficientes sobre el proxy, $\beta = 0, 0.1, 0.5, 2$, y un control que optimiza la recompensa real con $\beta = 0.1$.

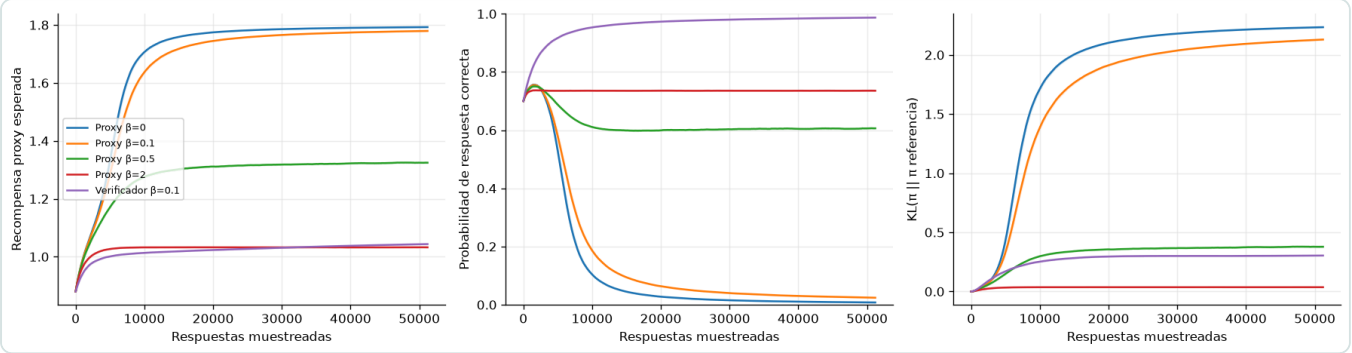
```
uv run python -m experimentos.ejecutar --capitulo 6
```

Métricas: recompensa proxy esperada, probabilidad de corrección y KL respecto a la referencia. Hay 800 actualizaciones de 64 muestras y 20 semillas. Guardamos también el óptimo analítico: el entrenamiento con presupuesto finito no tiene por qué alcanzarlo.

Resultado observado: sin KL, la corrección cae desde 70% hasta aproximadamente 0.78%, aunque el proxy mejora. Una KL fuerte reduce el desplazamiento, pero no corrige la preferencia equivocada del proxy. Optimizar el verificador lleva la corrección a aproximadamente 98.7%.

Modificación: intercambia la puntuación proxy de una respuesta correcta y la incorrecta, o cambia las probabilidades iniciales. Observa cómo cambian la solución analítica y la velocidad de aprendizaje.

Límite: es una distribución sobre cuatro respuestas fijas; no hay transformer, tokens, generalización entre prompts ni entrenamiento de un reward model. Ilustra RLOO, KL y mala especificación, no mide el comportamiento de un LLM real.



Para el grupo

1. ¿Qué diferencia hay entre un verificador correcto y uno que solo cubre algunos casos?
2. ¿Por qué una penalización KL no garantiza alineamiento?
3. ¿Cuándo deja de ser suficiente el modelo de bandit de un solo turno?

Conexión: la cobertura de datos, la exploración y el cambio de distribución llevan al capítulo 7.

Capítulo 7 · Otros temas de RL

Lectura: pp. 173–198; §§7.1–7.8. Capítulo original.

Idea central: el algoritmo depende también de cómo se obtiene información, de qué datos existen y de la escala temporal en la que se representa la tarea.

Resumen por bloques

§7.1 — Regret. En lugar de mirar solo el rendimiento final, medimos lo perdido mientras aprendemos frente a un comparador. En un entorno estático puede ser la mejor política fija; en uno cambiante puede ser una secuencia de políticas óptimas. Las garantías de regret dinámico necesitan restricciones sobre el cambio del entorno. El criterio bayesiano promedia incertidumbre con un prior; el minimax protege frente a una clase de casos. No son objetivos intercambiables.

§7.2 — Exploración. Una formulación bayesiana aumenta el estado con creencias sobre el modelo; planificar en ese espacio incorpora el valor de aprender. Salvo casos especiales, es caro. Thompson sampling muestrea un modelo de su posterior y actúa como si fuera correcto; UCB usa una estimación optimista. PSRL extiende el muestreo posterior a MDP y mantiene una política coherente durante un episodio. Sus garantías dependen del modelo, del prior o de los intervalos utilizados.

§7.3 — RL distribucional. Aprender la distribución del retorno aporta más información que aprender solo su esperanza. Puede representarse mediante cuantiles o masas categóricas. No equivale a estimar incertidumbre epistémica sobre Q: una distribución amplia puede deberse al azar del entorno. Tampoco basta discretizar una media escalar para afirmar que se ha aprendido toda la distribución de retornos.

§7.4 — Motivación intrínseca. La curiosidad recompensa novedad o aprendizaje del mundo; otros métodos recompensan adquirir habilidades y alcanzar metas. Bonos de conteo, RND, ganancia de información, empowerment, currículos y Go-Explore atacan problemas distintos. El ejemplo de la “televisión ruidosa” muestra que error de predicción alto no siempre significa información útil: un agente puede quedarse mirando ruido irreducible.

§7.5 — Jerarquía. Una opción es una política temporalmente extendida, con conjunto de iniciación y regla de terminación. El nivel superior escoge habilidades; la ejecución puede durar varios pasos. Option-critic, MAXQ y otras variantes distribuyen decisiones y crédito. En jerarquías feudales, un gestor propone metas a un ejecutor. El descubrimiento de subtareas busca cuellos de botella o habilidades diversas. Una jerarquía impuesta puede acelerar aprendizaje y también restringir las soluciones posibles.

§7.6 — Imitación. Behavior cloning trata acciones expertas como etiquetas. Su problema secuencial es que pequeños errores pueden llevar a estados ausentes de las demostraciones. IRL busca una recompensa que explique al experto, pero generalmente

no identifica una recompensa única. Los métodos de divergencia comparan distribuciones de ocupación estado-acción; GAIL y AIRL usan ideas adversarias.

§7.7 — Offline RL. Se aprende de un conjunto fijo sin poder probar nuevas acciones. La política puede elegir acciones sin soporte en los datos y explotar valores o modelos mal estimados. Restricciones de política, incertidumbre y penalizaciones conservadoras, como CQL, limitan este problema. Los modelos del mundo offline también requieren cautela ante predicciones fuera de distribución. Decision Transformer y modelos de trayectorias convierten partes del problema en modelado condicionado por retorno; pedir un retorno alto no garantiza que una acción lo cause en un mundo estocástico. Offline-to-online añade interacción posterior. DAgger corrige imitación pidiendo etiquetas expertas en estados visitados por el aprendiz: requiere interacción y acceso al experto.

§7.8 — RL general. AIXI combina una distribución sobre programas con planificación para definir un agente ideal. Es un referente teórico, no una receta ejecutable general: la mezcla universal no es computable en su forma ideal y la planificación es costosa. El cierre aborda agentes con capacidad y datos limitados.

Ecuaciones esenciales

En nuestro bandit, con medias de recompensa $\mu_t(a)$,

$$\mathcal{R}_T = \sum_{t=1}^{T} [\max_a \mu_t(a) - \mu_t(a_t)].$$

Es **pseudorregret**: usamos medias verdaderas para evaluar las acciones elegidas. El agente no conoce esas medias. En el escenario cambiante el comparador puede cambiar de brazo en cada instante.

Si una opción dura ι pasos, el objetivo temporal tiene la forma

$$y = \sum_{k=0}^{\tau-1} \gamma^k r_{t+k} + \gamma^\tau V(s_{t+\tau}).$$

Usar solo γ en el último término ignoraría su duración.

Experimento 7 · Explorar y adaptarse

Comparamos ϵ-greedy con media histórica, UCB1, Thompson Beta-Bernoulli y Thompson con olvido. Los cuatro brazos tienen probabilidades de éxito $(0.2, 0.4, 0.6, 0.8)$. En el segundo escenario se invierte ese orden después de 2.000 de las 4.000 interacciones.

Cada método prueba los cuatro brazos al inicio. UCB1 usa $\hat{\mu}(a) + \sqrt{2\log(t)/N(a)}$. Thompson estándar empieza con Beta(1,1). En la variante con olvido, los pseudoconteos por encima del prior se multiplican por 0.98 antes de incorporar cada observación. Esta última es una heurística adaptativa, no el posterior exacto del modelo estacionario.

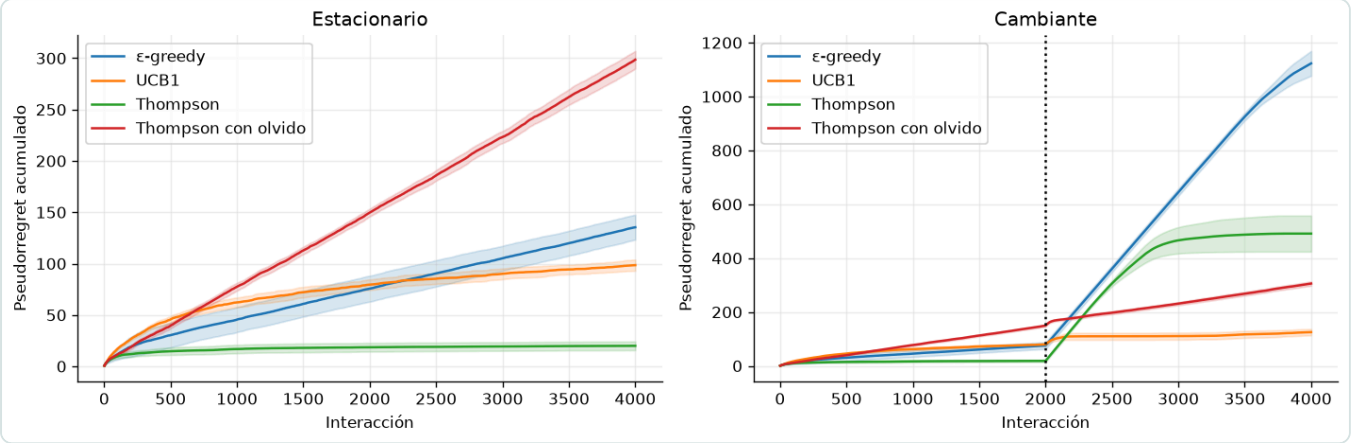
```
uv run python -m experimentos.ejecutar --capitulo 7
```

Métrica: pseudorregret acumulado, con 20 trayectorias independientes por método y escenario. El ruido de recompensas se comparte por réplica e instante entre métodos para facilitar la comparación; no se comparten las recompensas de acciones no elegidas.

Resultado observado: Thompson estándar funciona especialmente bien en el escenario fijo. Olvidar mejora su adaptación al cambio en este ejemplo, pero empeora mucho su rendimiento estacionario. UCB1 obtiene el menor regret cambiante entre las configuraciones probadas. El experimento no confirma un ganador universal ni que olvidar sea siempre mejor.

Modificación: cambia el instante de inversión, la separación entre brazos o el factor de olvido. Separa hiperparámetros de evaluación si quieres compararlos formalmente.

Límite: el laboratorio implementa §§7.1–7.2 y conecta con incertidumbre; los apartados de jerarquía, imitación y offline RL se resumen, pero no se reproducen aquí. Un bandit no prueba exploración profunda en MDP.



Para el grupo

1. ¿Por qué una incertidumbre predictiva grande puede ser inútil para explorar?
2. ¿Qué no puedes aprender de un dataset que nunca probó cierta acción?
3. ¿Qué supuestos hacen útil una habilidad aprendida al cambiar de tarea?

Conexión: vuelve a la pregunta inicial: ¿qué sabe el agente y qué datos puede conseguir?

Capítulo 8 · Agradecimientos y práctica de cierre

Lectura: p. 199. Apartado original.

Resumen fiel al apartado

Este capítulo reconoce la procedencia de parte del material de los capítulos 34 y 35 de *Probabilistic Machine Learning: Advanced Topics*, incluyendo trabajo escrito con Lihong Li. Murphy indica que esta monografía amplía y sustituye aquel tratamiento. También agradece comentarios de colegas y ayuda con figuras.

No introduce algoritmos ni resultados técnicos. La bibliografía que sigue tampoco es un capítulo de métodos. Para acompañar todos los capítulos numerados sin inventar contenido, añadimos una práctica metodológica propia, conectada con §1.3.7.

Experimento 8 · Elegir la mejor semilla fabrica una mejora

Pregunta: ¿qué sucede si comparamos la mejor ejecución de un algoritmo con el promedio de otro?

Simulamos 2.000 estudios. En cada uno, A y B tienen exactamente la misma distribución real de rendimiento, normal de media 0 y desviación 1, y disponemos de 20 semillas de cada algoritmo.

Comparamos tres formas de informar: media de A menos media de B; máximo de A menos media de B; y una nueva evaluación independiente del resultado seleccionado. En este modelo toda semilla comparte calidad real, así que seleccionar solo el ruido positivo no crea una mejora real.

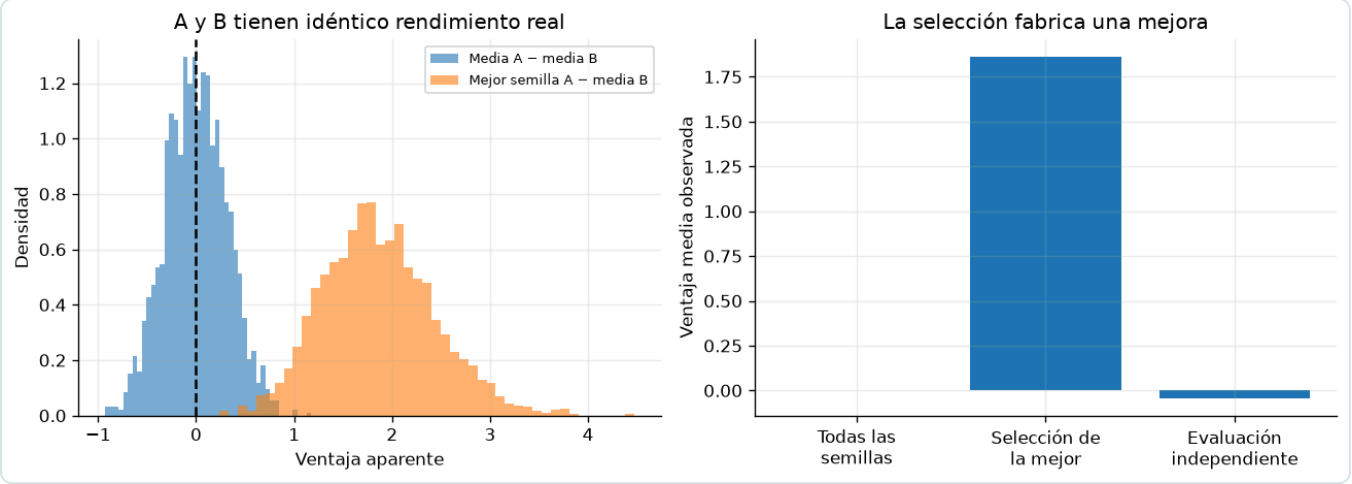
```
uv run python -m experimentos.ejecutar --capitulo 8
```

Hipótesis: comparar todas las semillas produce una diferencia cercana a cero. Seleccionar la mejor de A produce una ventaja positiva ficticia.

Métrica: distribución de la ventaja observada en los estudios. El resultado guardado muestra aproximadamente 0.001 al comparar medias, 1.864 al seleccionar la mejor semilla y −0.044 en la evaluación independiente. Son fluctuaciones de la simulación alrededor de las expectativas correspondientes.

Modificación: cambia 20 semillas a 5 y a 100. Cuantas más oportunidades de seleccionar un extremo, mayor puede ser el optimismo introducido por esa selección.

Límite: aquí las diferencias entre semillas son puro ruido, por construcción. En una tarea real puede existir calidad persistente entre políticas; la evaluación independiente sigue siendo necesaria para distinguirla del ruido de selección.



Para cerrar el grupo

Lleva una pequeña ficha con el entorno, la recompensa, la política, el presupuesto de interacción y cómputo, las semillas y las métricas. Añade una afirmación respaldada por tus datos y otra que tus datos no permiten sostener. Cita al autor del libro y distingue los experimentos de esta guía de las figuras o resultados originales.

Plan de lectura y glosario

Este calendario es una propuesta adaptable. Cada sesión puede durar 75–90 minutos: 10 de repaso, 25 de explicación, 25 de experimento y 15–30 de discusión. Los capítulos largos merecen más de una sesión.

Propuesta de 14 sesiones

Sesión	Lectura	Preparación concreta
0	Conocimientos previos	Resolver el autodiagnóstico y abrir el cuaderno 1
1	Cap. 1	Describir un problema propio como agente, observación, acción y recompensa
2	§§2.1–2.3	Derivar Bellman para una cadena de tres estados
3	§§2.4–2.5	Ejecutar Q-learning y explicar SARSA, replay y red objetivo
4	§§3.1–3.2	Explicar la identidad del gradiente y ejecutar el baseline
5	§§3.3–3.6	Comparar PPO, TRPO y SAC por objetivo, datos y restricciones
6	§§4.1–4.3	Ejecutar Dyna y contar datos reales y actualizaciones
7	§§4.4–4.5	Comparar reconstrucción, valor y autopredicción; discutir transferencia
8	Cap. 5	Ejecutar el juego y diferenciar media temporal de última política
9	§§6.1–6.2	Ejecutar el proxy; distinguir recompensa, preferencias y KL
10	§§6.3–6.4	Dibujar un ciclo de LLM, herramienta, observación, recompensa y actualización
11	§§7.1–7.4	Comparar regret fijo y dinámico; ejecutar los bandits
12	§§7.5–7.8	Contrastar opciones, imitación, offline RL y límites computacionales
13	Cap. 8 y cierre	Ejecutar selección de semillas y presentar una variante propia

Para una primera pasada corta, dedica una sesión a cada capítulo 1–7 y otra al cierre. Reserva las pruebas técnicas y las familias de algoritmos menos familiares para la segunda pasada.

Ficha reutilizable para preparar una sesión

1. **Problema:** ¿qué asume este capítulo sobre entorno, acciones, observaciones y datos?

2. **Objeto aprendido:** ¿valor, política, modelo, recompensa o representación?
3. **Una ecuación:** explica qué representa cada término y de dónde se obtienen sus datos.
4. **Una comparación:** identifica qué cambió y qué presupuesto se mantuvo fijo.
5. **Una evidencia:** anota una métrica con variabilidad.
6. **Una limitación:** escribe qué no permite concluir el experimento.
7. **Una pregunta:** formula algo que otro miembro pueda discutir o comprobar.

Glosario de consulta

Término	Significado en esta guía
Política	Regla o distribución que elige acciones
Retorno	Suma de recompensas, posiblemente descontadas
Valor	Esperanza del retorno bajo una política
Ventaja	Diferencia entre el valor de una acción y el valor medio del estado
Bootstrap	Usar una estimación aprendida dentro del objetivo de otra actualización
On-policy	Aprender sobre la política que genera los datos usados
Off-policy	Aprender sobre una política distinta de la que generó los datos
Offline RL	Aprender de un dataset fijo sin nueva interacción
Replay buffer	Memoria de transiciones reutilizables
Red objetivo	Copia o promedio lento que define objetivos de aprendizaje
Actor / crítico	Política / estimador de valor
Modelo del mundo	Predictor de consecuencias de acciones
Planificación	Usar un modelo para comparar decisiones o mejorar una política
MCTS	Búsqueda de árbol guiada mediante simulaciones
MPC	Optimizar un plan, ejecutar su primera acción y replanificar
No estacionariedad	Cambios en la distribución o dinámica que enfrenta el aprendiz
Mejor respuesta	Estrategia óptima frente a oponentes fijos
NashConv	Suma de las mejoras unilaterales posibles
CTDE	Entrenamiento centralizado y ejecución descentralizada
RLHF / RLAIF	Aprendizaje con feedback humano / de otro modelo
RLVR	Aprendizaje con recompensas verificables
RLOO	REINFORCE con baseline que excluye la propia muestra
DPO	Optimización directa a partir de pares de preferencias
Regret	Pérdida frente a un comparador definido
Pseudorregret	Diferencia acumulada entre medias de recompensa
Opción	Habilidad con iniciación, política interna y terminación
Behavior cloning	Imitación por aprendizaje supervisado de acciones
IRL	Inferir recompensas a partir de comportamiento

Término	Significado en esta guía
CQL	Aprendizaje de Q con una penalización conservadora
AIXI	Definición teórica de un agente con mezcla universal y planificación

Pregunta transversal

En cada capítulo, identifica quién paga el coste de la incertidumbre: ¿se paga con más interacciones, más cómputo, supuestos más fuertes, datos expertos o restricciones sobre la política? Esa pregunta conecta casi todo el libro.

Referencia: organización pedagógica propia a partir del índice de Murphy v5.

Protocolo de reproducción e interpretación

Qué se ejecutó

La referencia usa semilla base 2026 y 20 réplicas. Los módulos 1–6 repiten con semillas 2026–2045; el módulo 7 genera 20 trayectorias por lote a partir de flujos pseudoaleatorios separados para decisiones y recompensas. El módulo 8 genera 2.000 estudios sintéticos, cada uno con 20 semillas de A y B.

Los presupuestos, tasas y demás hiperparámetros están en el código y en cada resumen.json. El manifiesto de resultados registra versiones, tiempos por capítulo y hashes SHA-256 del código usado. uv.lock fija la resolución del entorno; requirements.txt fija las dependencias de ejecución para la alternativa pip.

Las réplicas miden azar en observaciones, acciones o inicializaciones según el experimento. No representan una muestra de todos los entornos posibles.

Archivos que puedes inspeccionar

Archivo por capítulo	Contenido
figura.png	Gráfica para lectura y presentación
figura.svg	Figura vectorial para exportar
datos.npz	Arrays por réplica y punto temporal; no solo promedios
metricas.csv	Métricas finales por réplica o por estudio
resumen.json	Parámetros principales y agregados

Los datos se abren con numpy.load("resultados/cap02/datos.npz"). Usa .files para ver las claves. Las trayectorias completas de estados y acciones no se guardan; sí las series de métricas necesarias para reconstruir las figuras.

Bandas y evaluación

Las bandas representan media ± 1.96 veces el error estándar entre ejecuciones. Son aproximaciones normales, con solo 20 réplicas por defecto, y se calculan punto a punto. No son desviaciones estándar individuales ni intervalos simultáneos. Un IC puede salir del rango de una métrica acotada; no lo truncamos para aparentar mayor precisión.

No suavizamos las curvas. Los ejemplos pequeños permiten evaluación exacta en los capítulos 2, 3, 5 y 6. Dyna se evalúa mediante error respecto al óptimo conocido y metas conseguidas durante entrenamiento. El regret de bandits usa las medias verdaderas solo para evaluar.

Las configuraciones comparten la tarea y el presupuesto dentro de cada comparación. Igual número de interacciones no equivale a igual cómputo: Dyna registra ambos. Compartir semillas o ruido no implica que las trayectorias sean iguales cuando las políticas divergen.

Validación numérica

Las pruebas contrastan la implementación con propiedades independientes: solución cerrada de una cadena determinista, residuos de Bellman, valores terminales, invariancia de softmax, gradientes por diferencias finitas, solución analítica regularizada, esperanza exacta de RLOO, equilibrio del juego, reproducibilidad y cotas del pseudorregret.

La validación documental comprueba presencia de capítulos, cuadernos, resultados, fórmulas renderizadas y destinos locales de enlaces. Ejecutar los cuadernos de principio a fin comprueba además imports y rutas.

Cómo proponer una variante

Antes de ejecutar, escribe una hipótesis y cambia un solo factor: descuento, exploración, ruido, planificación, penalización KL o cambio del entorno. Elige un directorio nuevo con --salida. Mantén un conjunto de semillas para explorar hiperparámetros y otro para evaluarlos si pretendes hacer una comparación formal. Informa resultados desfavorables y diferencias en cómputo.

Los laboratorios son demostraciones didácticas. No justifican conclusiones sobre Atari, robótica o LLM reales sin experimentos adicionales.

Resultados observados

Ejecución de referencia: semilla base 2026, 20 réplicas. Los valores con ± indican media y semianchura del IC normal aproximado del 95%. Las cifras se generan a partir de los JSON; no son resultados del libro.

Cap.	Métrica	Resultado
1	Acierto sin memoria	0.4955 ± 0.0094
1	Acierto con memoria	1.0000 ± 0.0000
2	RMSE final de Q	0.0158 ± 0.0032
2	Valor de política final	0.8166 ± 0.0000
3	Retorno sin baseline, restando 5	0.9680 ± 0.0018
3	Retorno con baseline, restando 5	0.9673 ± 0.0006
4	RMSE con 0 pasos de planificación	0.0186 ± 0.0032
4	RMSE con 5 pasos de planificación	0.0000 ± 0.0000
5	NashConv de políticas actuales	1.6699 ± 0.1049
5	NashConv de políticas medias	0.0310 ± 0.0006
6	Corrección, proxy sin KL	0.0078 ± 0.0001
6	Corrección, proxy con β=2	0.7358 ± 0.0002
6	Corrección, verificador con β=0.1	0.9871 ± 0.0001

Varianza del gradiente

Con política fija, la traza de covarianza baja de 17.067 a 0.392. La reducción de varianza no produce una mejora final clara en la recompensa de este ejemplo.

Exploración y cambio del entorno

Método	Regret estacionario	Regret cambiante
ε-greedy	135.3900 ± 12.1126	1123.0300 ± 46.1016
UCB1	98.5100 ± 5.4502	125.3400 ± 11.8200
Thompson	19.9600 ± 4.2416	491.0400 ± 66.8177
Thompson con olvido	298.5500 ± 8.5112	305.7400 ± 7.8340

Thompson estándar destaca en el entorno fijo. El olvido ayuda a Thompson en el cambiante, pero UCB1 logra aquí el menor regret cambiante. No hay un ganador universal.

Sesgo de selección

Con algoritmos de igual calidad real, comparar medias produce 0.0014; seleccionar la mejor semilla de A produce 1.8640; evaluar de nuevo produce -0.0444. Son promedios de 2.000 estudios sintéticos.

Tiempo y alcance

Los ocho módulos tardaron 13.28 segundos en esta máquina, sin incluir instalación, creación de cuadernos ni validación. El tiempo en otro equipo puede variar.

Dyna compara presupuestos iguales de interacción, con cómputo diferente. El laboratorio de LLM es categórico. Las curvas no establecen resultados generales ni reproducen benchmarks de redes profundas.

Consulta los JSON y arrays por capítulo para más métricas y precisión. No se ha elegido solo la mejor semilla.

Fuente, versión y atribución

Obra: Kevin P. Murphy, *Reinforcement Learning: An Overview*, arXiv:2412.05265v5.

- Ficha de la versión exacta.
- PDF de la versión exacta.
- HTML para navegar por secciones.
- Licencia CC BY 4.0.

La ficha de arXiv indica revisión del 1 de diciembre de 2025. El PDF descargado tiene fecha de portada del 3 de diciembre de 2025 y 253 páginas. La versión HTML consultada muestra otra fecha de portada; para fijar versión y paginación usamos el PDF con sufijo v5. Las páginas indicadas son las numeradas en el PDF y coinciden con su posición en el archivo descargado.

El archivo PDF conserva el original descargado. metadata.json registra URLs, fecha de consulta y SHA-256. No hemos incorporado versiones posteriores ni tratado las afirmaciones sobre el estado del arte del libro como resultados actuales comprobados independientemente.

La guía es una adaptación resumida en español con explicaciones, ejercicios, código y figuras propios. Se han reformulado conceptos, unificado la notación y añadido condiciones cuando una fórmula requiere supuestos. Los ejemplos pequeños no son experimentos originales del autor. El capítulo 8 solo contiene agradecimientos; su práctica metodológica es un añadido vinculado a §1.3.7.

Se conserva la atribución al autor y la referencia a la licencia del original. Este material no implica que Kevin Murphy lo haya revisado o respaldado.

Fuente: Kevin P. Murphy · arXiv:2412.05265v5 · CC BY 4.0. Guía adaptada en español; sin respaldo implícito del autor. Fórmulas MathML y figuras incluidas para lectura sin conexión.